KEEPING DATASET BIASES OUT OF THE SIMULATION

A Debiased Simulator for Reinforcement Learning based Recommender Systems

Jin Huang · Harrie Oosterhuis · Maarten de Rijke · Herke van Hoof

University of Amsterdam · RecSys '20, 14th ACM Conference on Recommender Systems

Presentado por: Darío Donoso y Josefina Nazar

Reinforcement Learning en Recomendación (RL4Rec)

- RL para recomendación maximiza la recompensa acumulada en varias interacciones: clicks totales, dwell time por sesión, etc.
- RL aprende explorando. Es decir, recomienda un ítem y recibe feedback.
- Online puede significar recomendar malos ítems y perjudicar la experiencia del usuario.

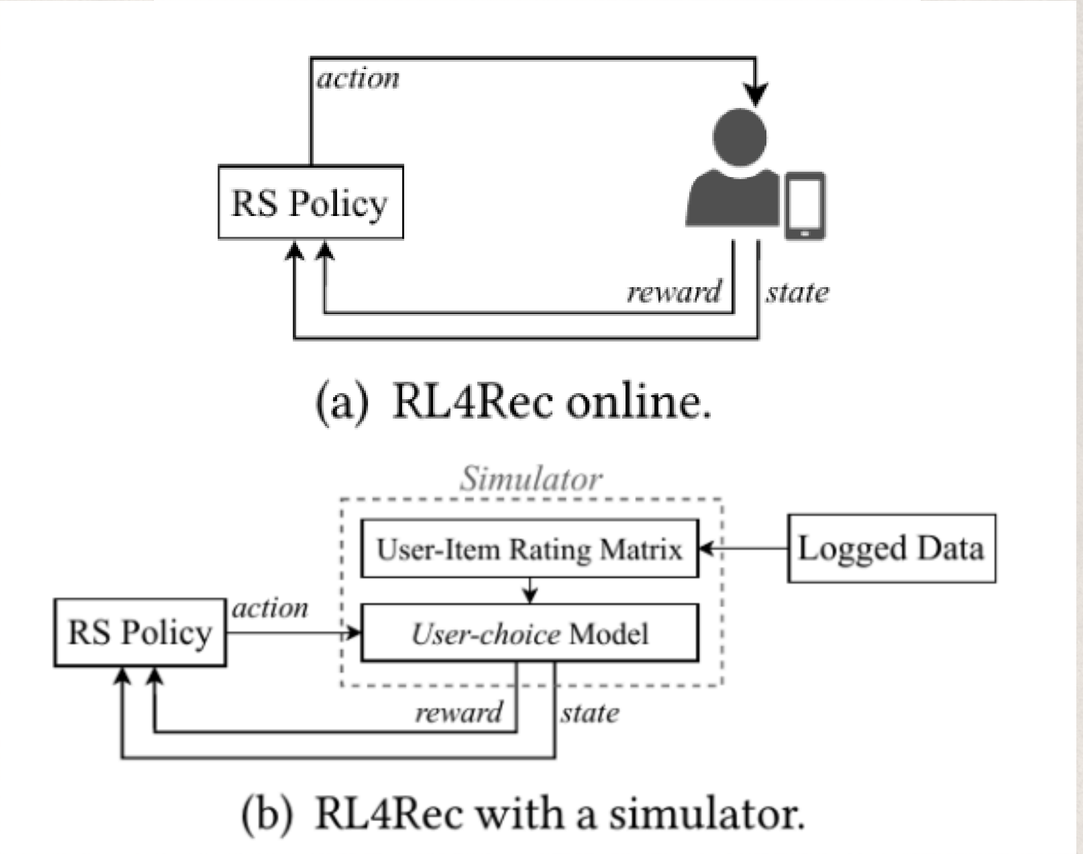


Fig. 1. Huang et al., RecSys '20, 2020

Reinforcement Learning en Recomendación (RL4Rec)

Se simula el comportamiento del usuario:

- Simuladores para un dataset específico.
- Simuladores con datos sintéticos, simplifican demasiado el comportamiento.
- **Simuladores basados en datos registrados (logged data):** realistas, son los más usados.

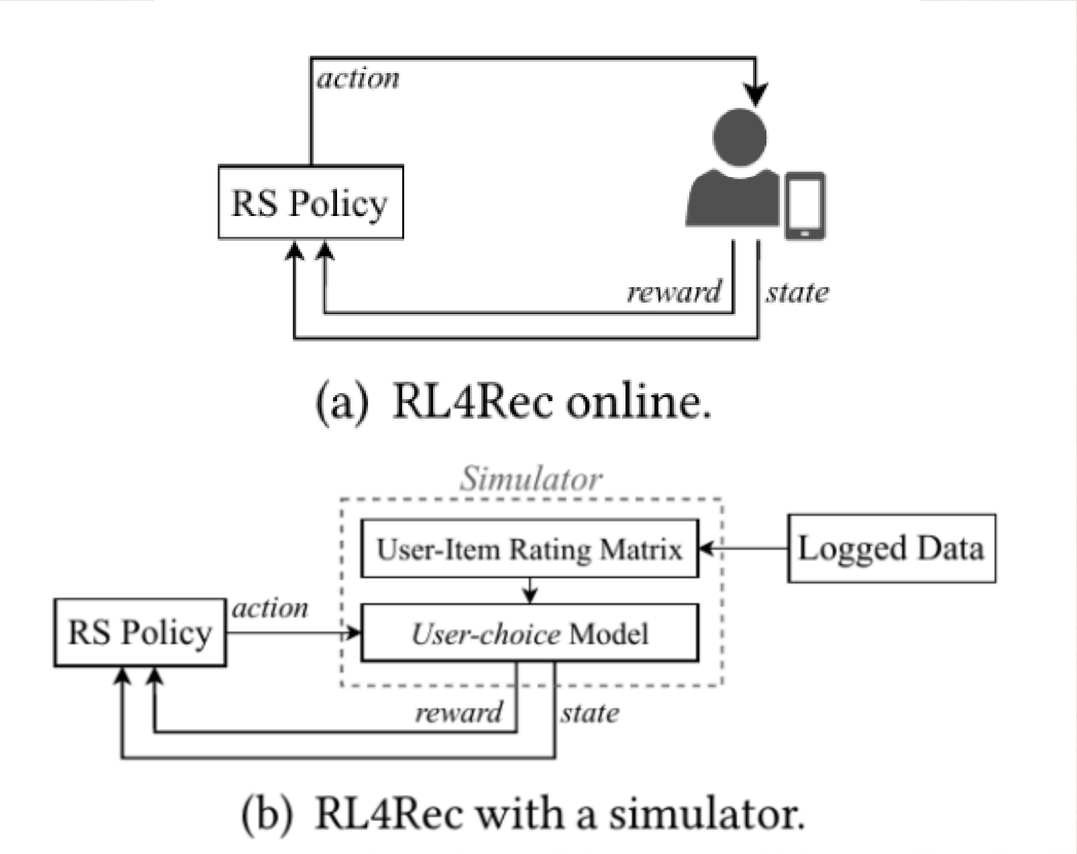


Fig. 1. Huang et al., RecSys '20, 2020

¿Cómo se Construye un Simulador a Partir de Logs?

01 Se recopilan datos históricos (logs), con los ratings que los usuarios dieron en el pasado. La matriz usuario–ítem suele ser *sparse*.

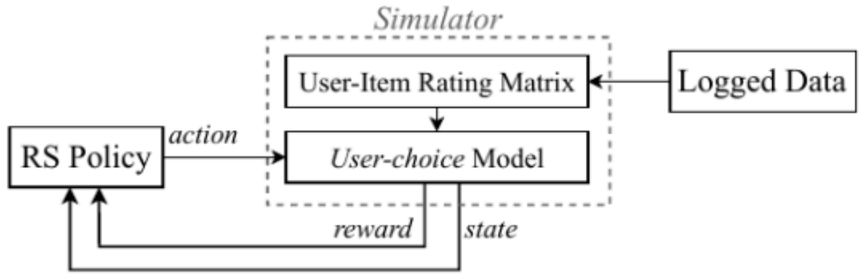
02 Se entrena MF sobre esos datos, y se completa la matriz de ratings Ŷ.

03 Se define el User-Choice Model, donde se define que el usuario hace click si el rating predicho es mayor a un umbral T.

$$f_t = \mathbf{1}\left[\hat{y}_{u,\ i_t} > T\right]$$

04 La política recomienda un ítem, recibe click o skip como recompensa y aprende. Se repite turno a turno.

Click: Recompensa +x
Skip: Recompensa -y.
K turnos por episodio.



(b) RL4Rec with a simulator.

Fig. 1b. Huang et al., RecSys '20, 2020

Limitación 1: los Datos están Sesgados (MNAR)

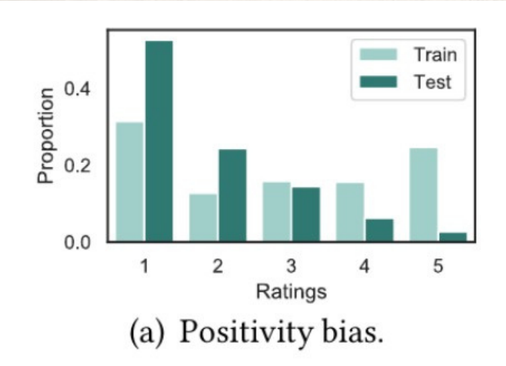
MNAR: observar un rating depende del rating

$$P(y_{u,i},\, o_{u,i}) = P(o_{u,i} \mid y_{u,i})\; P(y_{u,i})$$

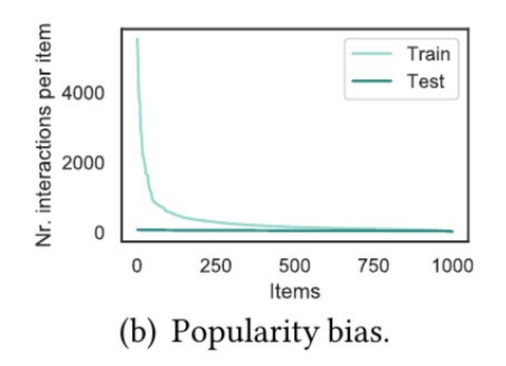
Sin sesgo (MCAR): todos igual de probables

$$P(o_{u,i}) = P(o_{u',i'}) \quad \forall\, (u,i),\, (u',i')$$

o = 1 si el rating del par (u, i) aparece en los logs



(a) Positivity bias.



(b) Popularity bias.

Fig. 2. Huang et al., RecSys '20, 2020

Positividad

Los usuarios tienden a calificar solo lo que les gusta: los ratings altos quedan sobrerrepresentados.

$$y_{u,i} > y_{u,i'} \;\Rightarrow\; P(o_{u,i}) > P(o_{u,i'})$$

Popularidad

Los usuarios interactúan más con ítems populares: muchas interacciones se concentran en pocos ítems.

$$\mathrm{pop}(i) > \mathrm{pop}(i') \;\Rightarrow\; P(o_{u,i}) > P(o_{u,i'})$$

Limitación 2: ¿Cómo se Evalúa el Simulador?

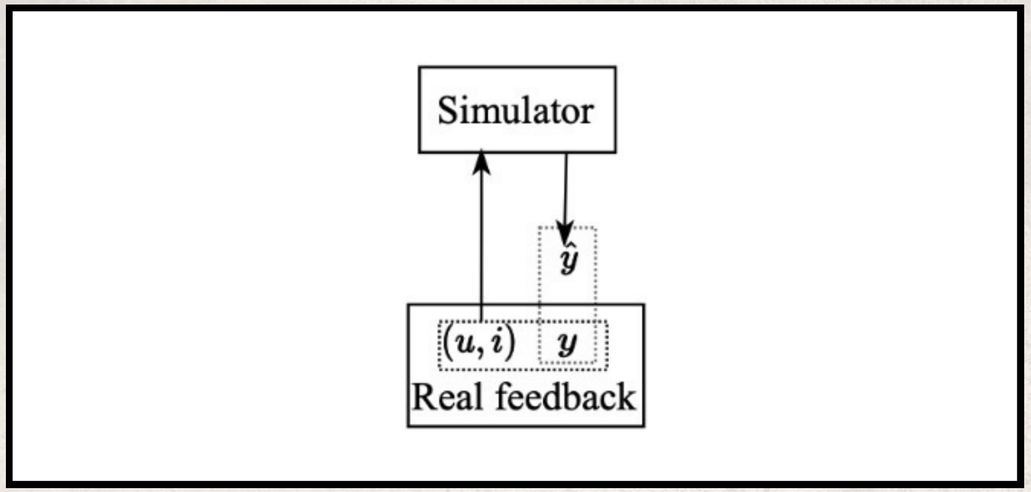
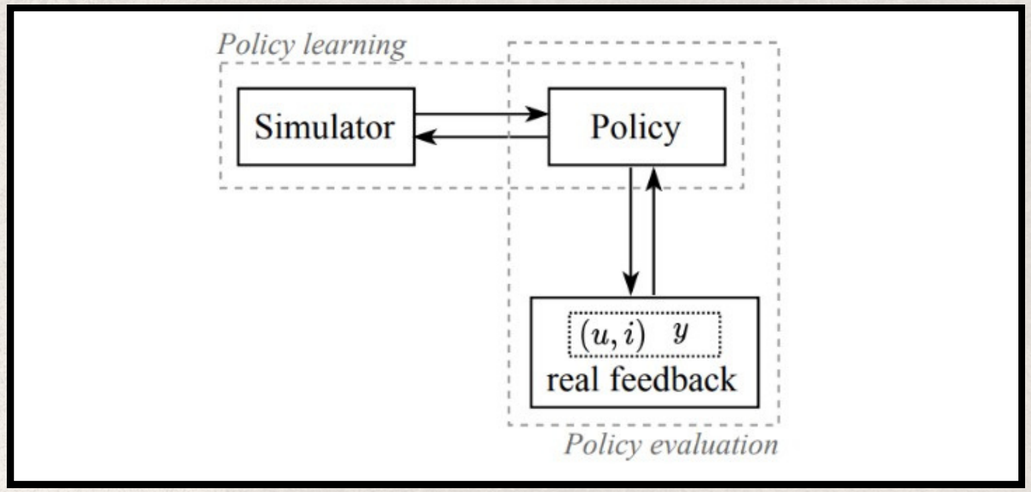


Fig. 4a Huang et al., RecSys '20, 2020

- Evaluación según comportamiento observado del usuario



Presentación RecSys '20, Diapositiva 9, Huang et al., 2020

- Evaluación según desempeño de la política, feedback real del usuario.

Recomendación Secuencial como MDP

s Historial de ítems recomendados y su feedback. Inicia vacío.

$$s_t^u = ([i_1, i_2, \ldots, i_t] , [f_1, f_2, \ldots, f_t])$$

$$f_t \in \{0, 1\}$$

a Acción, un ítem por recomendar según la política π aprendida por DQN.

$$a_t = i_t$$

r Recompensa, función según click o skip. En el paper, +1 por clic y -2 por skip.

$$r_t = R(f_t)$$

γ Factor de descuento, cuánto peso se le asigna al futuro. En el paper, γ = 0,9 (Zou et al. 2019)

$$\gamma \in [0,1]$$

Obj. $\pi^* = \underset{\pi}{\operatorname{argmax}}\ \mathbb{E}[\Sigma^K_{t=1}\ \gamma^{t-1}\ r_t]$

Maximizar recompensa acumulada.

Recomendaciones Secuenciales Sesgadas

- La función de pérdida de MF, naive, es sesgada.

$$\mathcal{L}_{Naive} = \frac{1}{|\{(u,i) : o_{u,i} = 1\}|} \sum_{(u,i) :\, o_{u,i} = 1} \delta_{u,i}(Y, \hat{Y})$$

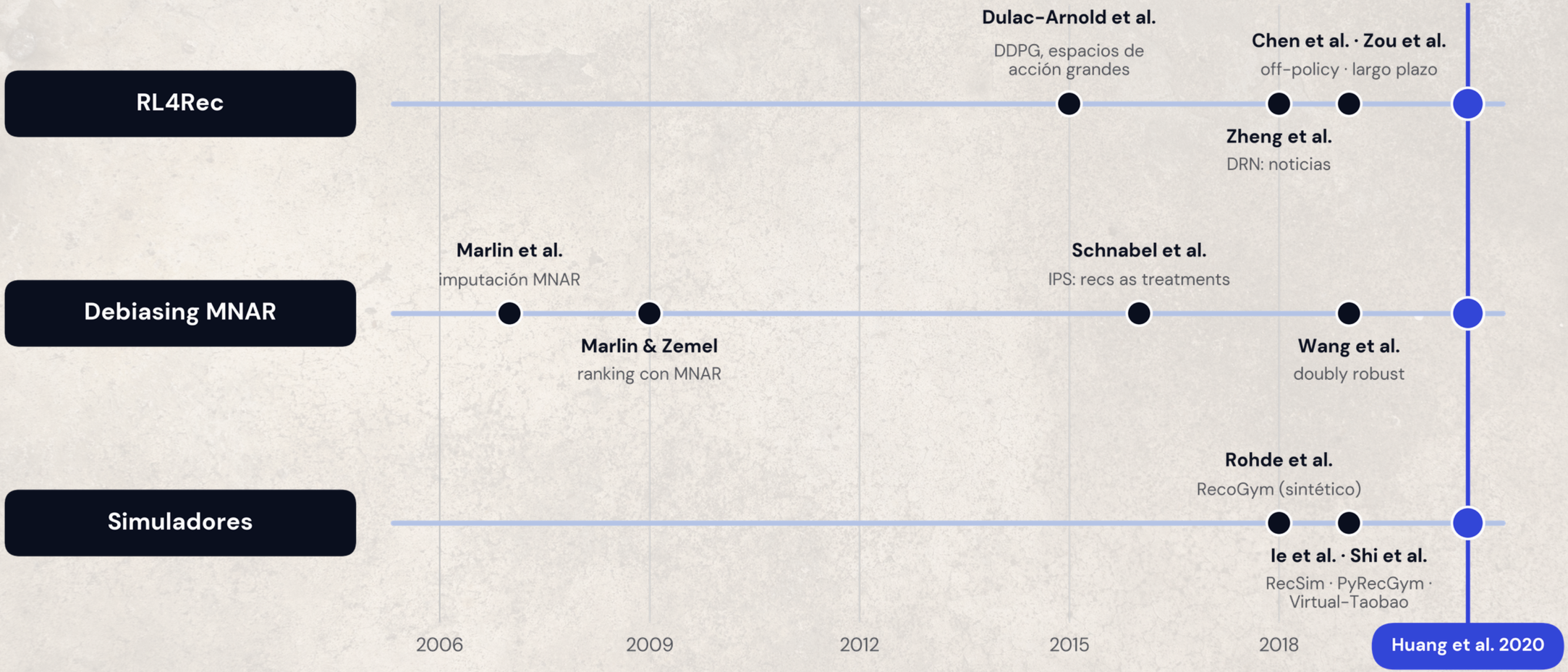
$\delta = (y - \hat{y})^2$ para MSE o $|y - \hat{y}|$ para MAE

$$\mathbb{E}[\mathcal{L}_{Naive}] = \frac{\sum_{u,i} P(o_{u,i} = 1)\, \delta_{u,i}}{\sum_{u,i} P(o_{u,i} = 1)} \neq \frac{1}{NM} \sum_{u,i} \delta_{u,i}$$

- MF-Naive trata todos los ratings observados como si fueran MCAR (Missing Completely at Random)
- Los pares con P(o = 1) alta (ratings altos, ítems populares) dominan el entrenamiento
- Resultado: $\hat{Y}$ sobreestima y el simulador hace clic de más.

En Yahoo!R3, el 24,6% de los ratings en los logs son 5 estrellas, en la muestra aleatoria, únicamente el 2,6%.

Estado del Arte



Estado del Arte · RL4Rec

Dulac-Arnold et al. (2015)

Introducción de DDPG. Extiende DQN para para espacios de acción de alta dimensión.

Zheng et al. (2018) · DRN

DQN para recomendar noticias, en línea con usuarios reales.

Zou et al. (2019)

Optimiza recompensas de largo plazo y entrena con un simulador aprendido de logs.

Chen et al. (2019)

REINFORCE off-policy en YouTube: corrige el sesgo de la política anterior.

Estado del Arte · Debiasing MNAR

Marlin et al. (2007)
Los ratings faltantes no son aleatorios (MNAR).

Schnabel et al. (2016)
Recomendación como tratamiento médico:

IPS para entrenar y evaluar sin sesgo.

Wang et al. (2019)
Doubly robust: imputación + IPS para reducir

la varianza.

Estado del Arte · Simuladores

RecoGym(2018) · RecSim (2019)

Simuladores sintéticos y configurables.

PyRecGym (2019)

Gym sobre datasets reales (logs).

Virtual-Taobao (2019)

Usuarios virtuales aprendidos de logs reales de Taobao (e-commerce de origen chino).

Contribución

IBMS IBMS (Intermediate Bias Mitigation Step), un paso entre los datos logueados y el modelo de predicción de ratings que mitiga el sesgo. Utiliza MF-IPS (Inverse Propensity Scoring). Aplicable a cualquier simulador genérico.

SOFA (Simulator for OFfline leArning and evaluation) es el primer simulador RL4Rec que corrige el sesgo antes de optimizar y evaluar. Código público.

EVAL Un método de evaluación nuevo: medir el efecto del sesgo mirando el desempeño (clics acumulados) de la política resultante, desplegada en un simulador insesgado construido con datos MCAR

Solución: IBMS

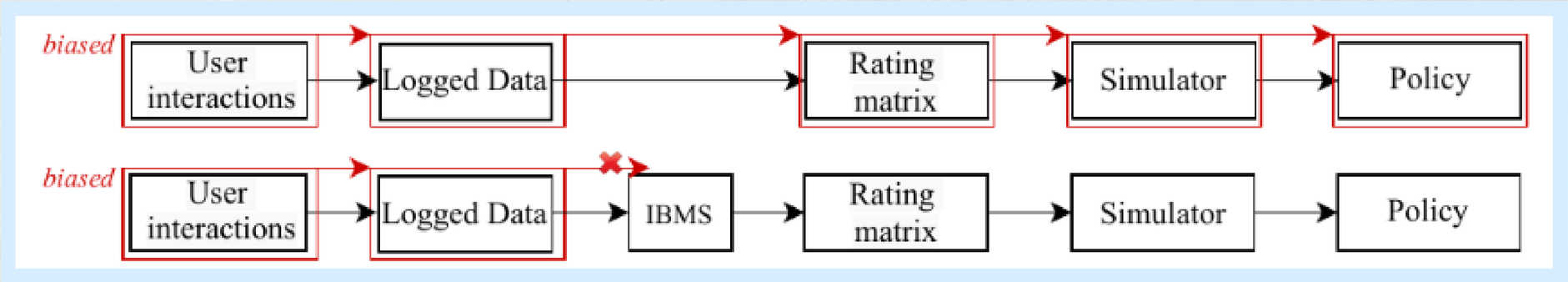


Fig. 3. Huang et al., RecSys '20, 2020

1. Estima P(o=1)

Para cada par (u,i), estima la probabilidad de que el ítem haya sido observado

2. Entrena con IPS-MF

Utiliza función de pérdida de Inverse Propensity Scoring (IPS). (Schnabel et al. 2016)

1. Estimar Probabilidad de Observación

Naive Bayes

$$P(o=1 \mid y) = \frac{P(y \mid o=1)\ P(o=1)}{P(y)}$$

P(y | o = 1): Distribución de ratings en los logs.

• P(o = 1): Fracción de la matriz que se observó.

• P(y): Distribución en la pequeña muestra MCAR (ítems al azar).

Yahoo!R3 – Sesgo de positividad

Regresión Logística

$$\hat{P}(o_{u,i}=1) = \sigma(\mathbf{w}^\top \mathbf{x}_{u,i} + b)$$

Predicción de pares (user, ítem) a partir de atributos.

Usuario: GÉNERO EDAD

ítem (abrigo): TIPO COLOR

Coat – Atributos usuario-ítem

2. Entrenar MF con IPS–MF

Pérdida Ingenua - MF NAIVE

$$\mathcal{L}_{Naive} = \frac{1}{|\{(u,i) : o_{u,i} = 1\}|} \sum_{(u,i) :\, o_{u,i} = 1} \delta_{u,i}(Y, \hat{Y})$$

$$\mathbb{E}[\mathcal{L}_{Naive}] = \frac{\sum_{u,i} P(o_{u,i} = 1)\, \delta_{u,i}}{\sum_{u,i} P(o_{u,i} = 1)} \neq \frac{1}{NM} \sum_{u,i} \delta_{u,i}$$

Pérdida IPS - MF IPS

$$\mathcal{L}_{IPS} = \frac{1}{NM} \sum_{(u,i) :\, o_{u,i} = 1} \frac{\delta_{u,i}(Y, \hat{Y})}{P(o_{u,i} = 1)}$$

$$\mathbb{E}[\mathcal{L}_{IPS}] = \frac{1}{NM} \sum_{u,i} \frac{P(o_{u,i} = 1)\, \delta_{u,i}}{P(o_{u,i} = 1)} = \frac{1}{NM} \sum_{u,i} \delta_{u,i}$$

Función de pérdida utilizada: Inverse Propensity Scoring (IPS). (Schnabel et al. 2016)

Solución: SOFA

Simulator for OFline leArning and evaluation
SOFA = matriz de ratings insesgada (MF-IPS) + User-choice Model

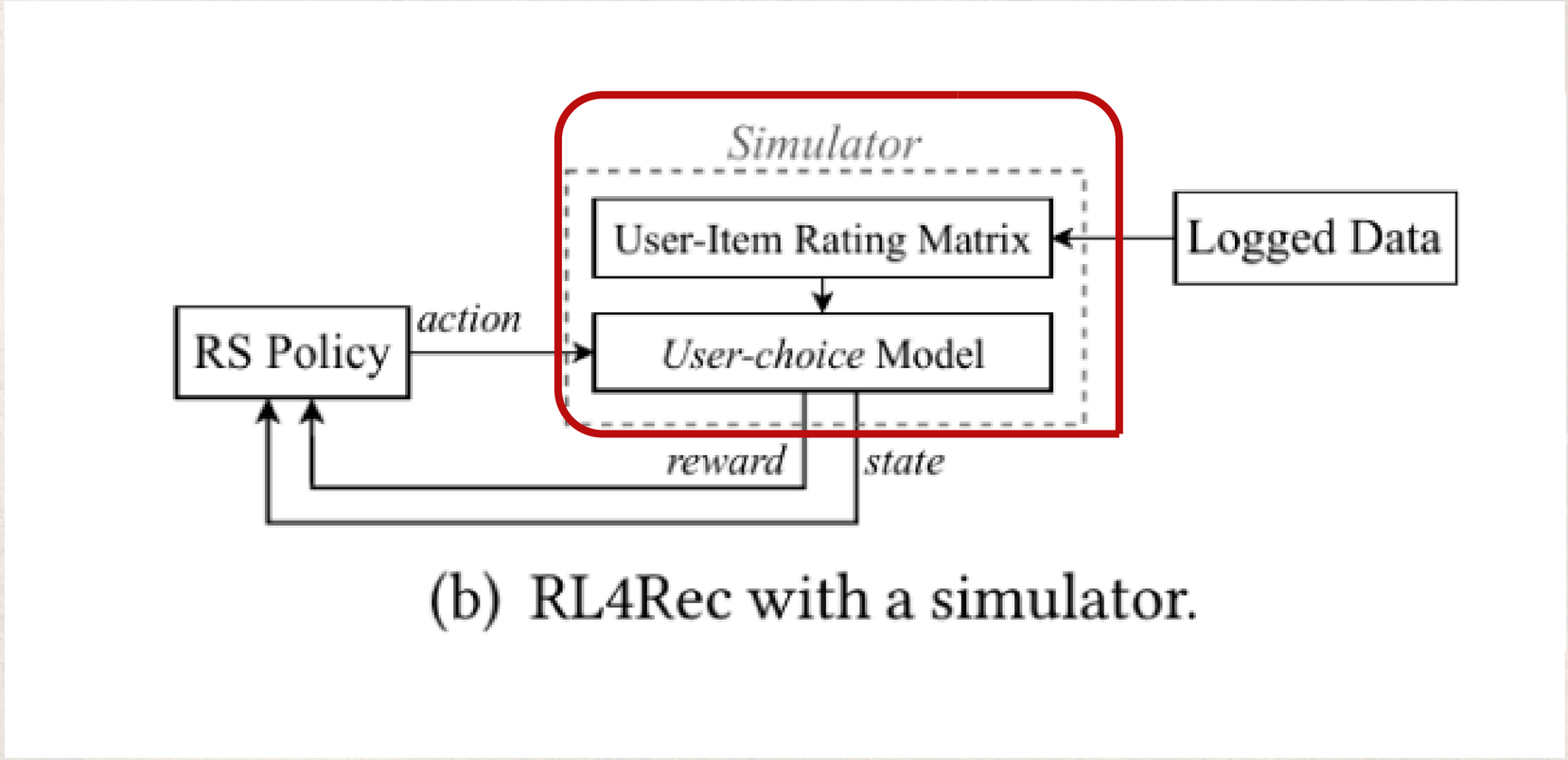


Fig. 1b. Huang et al., RecSys '20, 2020

Solución: SOFA

SOFA = matriz de ratings insesgada (MF-IPS) + User-choice Model

Matriz de ratings insesgada (MF-IPS)

$$\hat{y}_{u,i} = \mathbf{p}_u^\top \mathbf{q}_i + a_u + b_i + c$$

$$\arg\min_{P,Q,A} \underbrace{\sum_{(u,i):\, o_{u,i}=1} \frac{\delta(y_{u,i}, \hat{y}_{u,i})}{P(o_{u,i}=1)}}_{\text{Pérdida MF IPS}} + \underbrace{\lambda(\|P\|_F^2 + \|Q\|_F^2)}_{\text{Regularización}}$$

User-choice Model

Feedback

$$f_t = \begin{cases} 1\ (\text{click}) & \text{si } \hat{y}_{u,i_t} > 3 \\ 0\ (\text{skip}) & \text{e.o.c.} \end{cases}$$

Recompensas

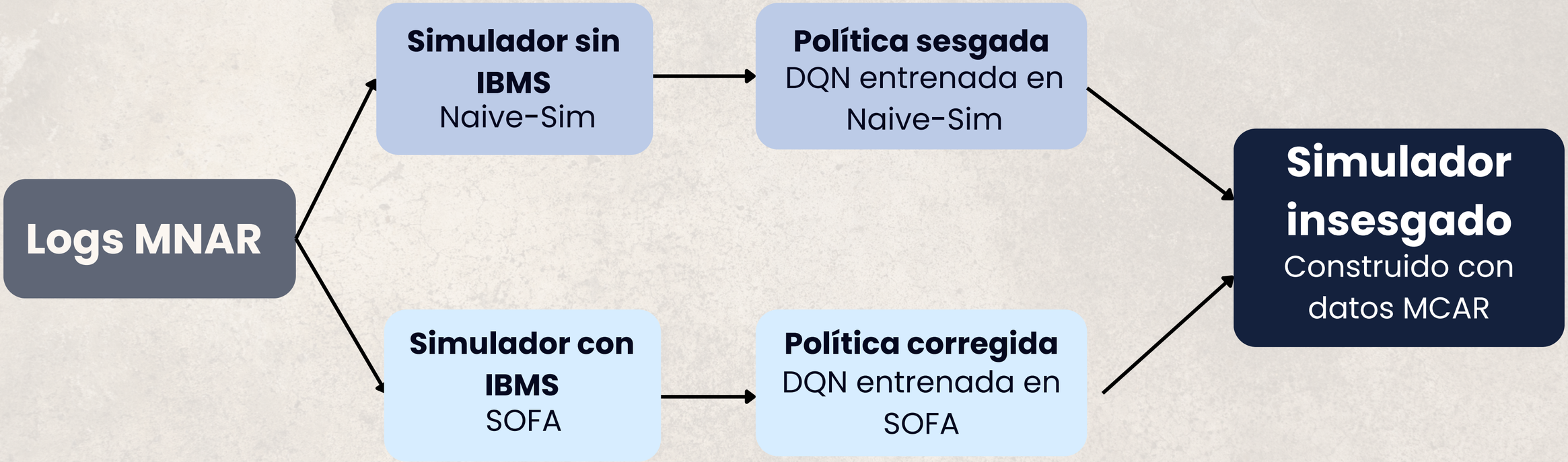
$$r_t = \begin{cases} +1 & \text{si } f_t = 1\ (\text{click}) \\ -2 & \text{si } f_t = 0\ (\text{skip}) \end{cases}$$

Transiciones

$$s_{t+1} = \left[(i_1, f_1), \ldots, (i_t, f_t), (i_{t+1}, f_{t+1})\right]$$

K = 10 turnos

Protocolo de Evaluación



Protocolo de Evaluación – Missing Values

Datos **MCAR** son datos cuyo rating **no depende** ni de los gustos del usuario ni de su popularidad.

En la práctica se pide a una cantidad fija de usuarios calificar una cantidad de ítems elegidos al azar.

Como son escasos, se soluciona de dos maneras:

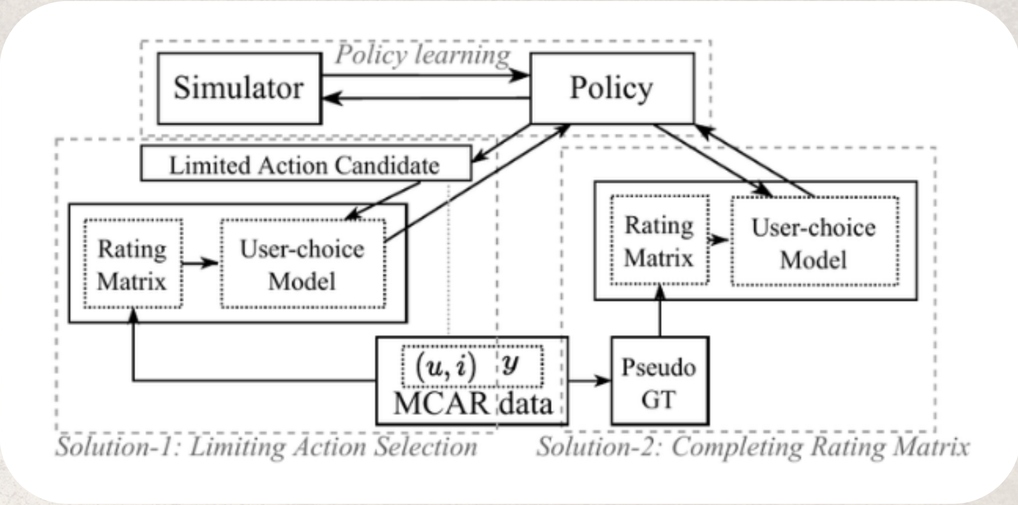


Fig. 4b. Huang et al., RecSys '20, 2020

Solución	1. **Limitar acciones**	2. **Completar Matriz**
Cómo	Solo se recomiendan ítems con rating MCAR para ese usuario.	MF sobre datos MCAR, que produce pseudo ground truth (Pseudo-GT).
Ventaja	Feedback siempre basado en ratings reales.	La política decide libremente.
Desventaja	Restringe la política, no evalúa su comportamiento real.	La Pseudo-GT es una estimación, no las preferencias reales.

Marco de Evaluación

RQ1 ¿El sesgo afecta al simulador?

RQ2 ¿IBMS lo mitiga?

RQ3 ¿Cómo influye la intensidad del sesgo?

Para responder las preguntas se utilizaron **dos datasets reales** y **uno sintético.**

Datasets

Yahoo!R3

Train (MNAR):
- 15.400 usuarios
- 1.000 ítems
- ~300.000 ratings

MCAR (Test):
- 5.400 usuarios x 10 ítems al azar

Propensity: Naive-Bayes

Coat

Train (MNAR):
- 290 usuarios
- 300 ítems
- 6.960 ratings, 24 por usuario

MCAR (Test):
- 290 usuarios x 16 ítems al azar

Propensity: Regresión Logística

Sintético

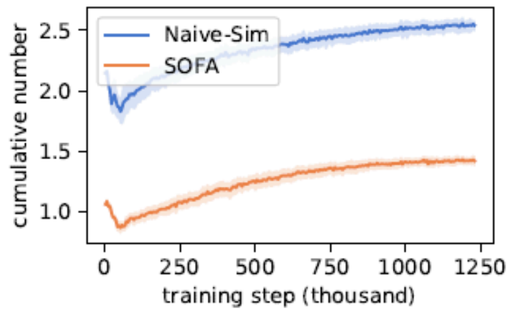
Train (MNAR):
- 300 usuarios
- 300 ítems
- 4.500 ratings, 5% del total de interacciones.
- Sesgo controlado por **α:**

$$P(o_{u,i} \mid y_{u,i}) = \alpha\, P(o_{u,i} \mid y_{u,i}, \text{pos}) + (1-\alpha)\, P(o_{u,i} \mid \text{unif})$$

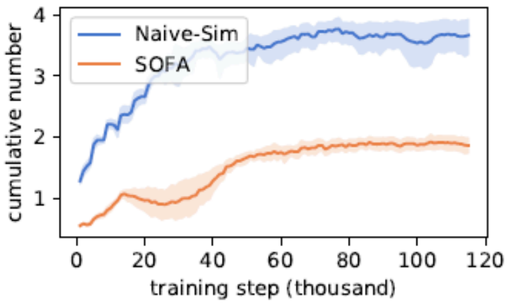
MCAR:
- Matriz completa construida con muestreo Normal.

Propensity: Conocida por construcción

1. ¿El Sesgo Afecta al Simulador?

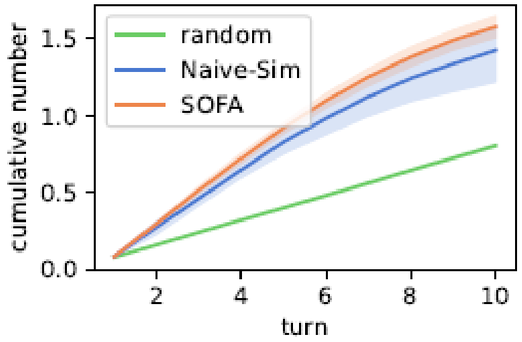


(a) Yahoo!R3.

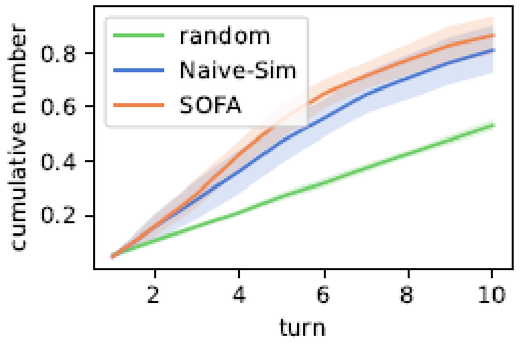


(b) Coat.

Fig. 7. Huang et al., RecSys '20, 2020



(b) Solution 2 on Yahoo!R3



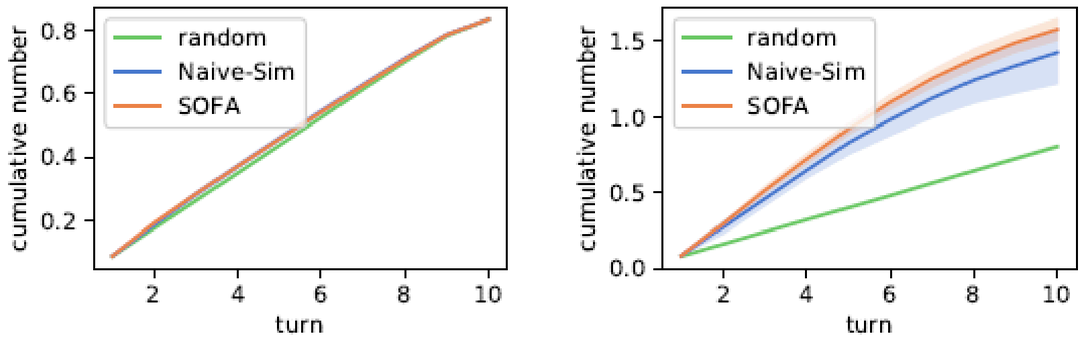
(d) Solution 2 on Coat

Fig. 9. Huang et al., RecSys '20, 2020

Se muestra la solución 2 para completar los datos MCAR (pseudo GT).

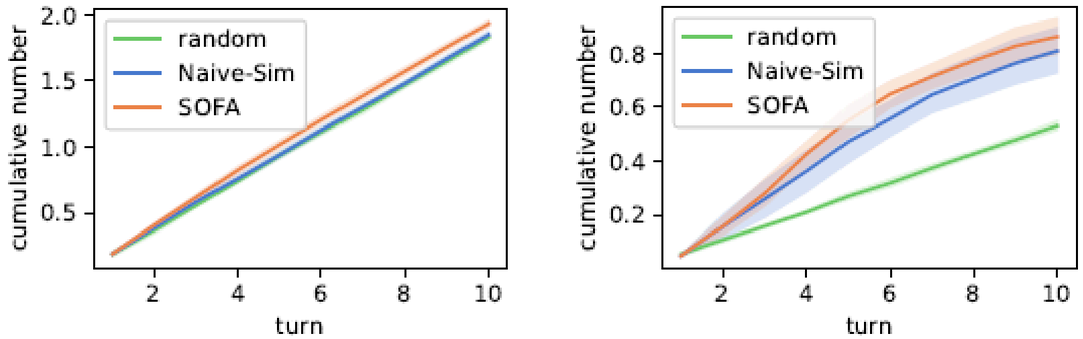
- Entrenando, la política de Naive-Sim recibe más clicks en ambos datasets.
- Al utilizar datos MCAR este orden se invierte.
- Naive-Sim es sobreestimado según su desempeño en entrenamiento.

2. ¿IBMS lo Mitiga?



(a) Solution 1 on Yahoo!R3

(b) Solution 2 on Yahoo!R3



(c) Solution 1 on Coat

(d) Solution 2 on Coat

Fig. 9. Huang et al., RecSys '20, 2020

Solución 1: Acciones Limitadas

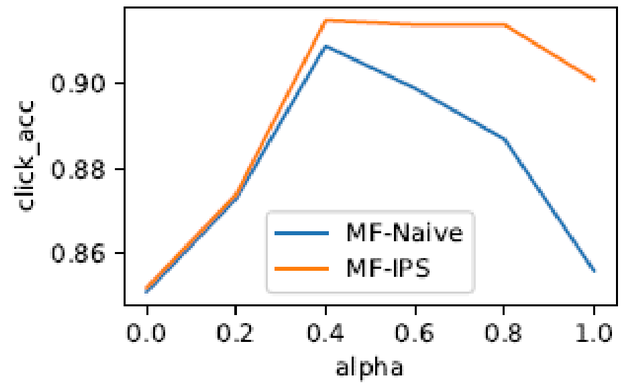
- No se observan diferencias significativas en cuanto al desempeño de la política resultante.

Solución 2: Completación de matriz

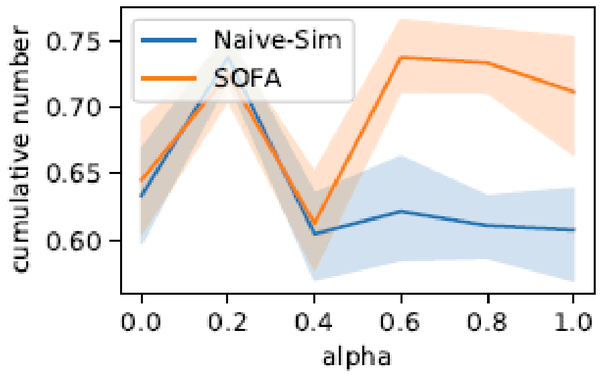
- Desempeño de SOFA superior a Naive-Sim en promedio (ambos datasets).
- Existe solapamiento de intervalos de confianza.

3. ¿Cómo Influye la Intensidad del Sesgo?

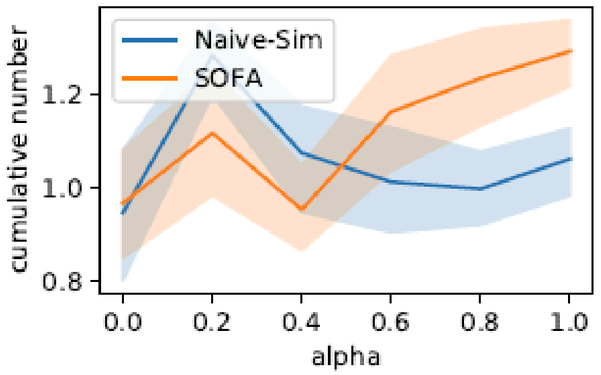
- **Sin sesgo (α = 0):** ambos métodos son equivalentes.
- **Sesgo alto (α ≥ 0,5):** MF-IPS / SOFA es consistentemente mejor.
- **Sesgo bajo (α < 0,5):** Considerando 10 turnos, SOFA puede tener un peor desempeño.



(a) Simulated feedback



(b) Clicks over 5 turns



(c) Clicks over 10 turns

Fig. 10. Huang et al., RecSys '20, 2020

Análisis Crítico

FORTALEZAS

- Modularidad de IBMS.
- Integración de medidas de nuevas evaluación a soluciones propuestas.
- Código open-source de SOFA.

DEBILIDADES

- Dependencia de datos MCAR de difícil acceso.
- Pseudo Ground Truth (GT) utilizado puede heredar errores.
- Decisiones sobre recompensas arbitrarias

PRINCIPALES VACÍOS

- No utilización de feedback implícito.
- No incorporan recomendaciones de múltiples ítems en simultáneo.
- Política construida sobre DQN, sin comparativa con otros modelos.

CONCLUSIONES

01 **El sesgo de los datos, potencialmente afecta el rendimiento de la política de recomendación, por lo que es relevante considerar su tratamiento.**

02 **La utilización de IBMS como proxy para la mitigación del sesgo, permite obtener mejoras observables cuanto a recompensas acumuladas de las políticas resultantes, sobre todo, cuando el sesgo en los datos es alto.**

03 **El rendimiento durante el entrenamiento no es confiable por sí solo, ya que depende de factores como el sesgo de los datos del simulador. Identificarlos es clave para diseñar metodologías que lleven a mejores políticas en un contexto real.**

Referencias

Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., & Chi, E. H. (2019). Top-k off-policy correction for a REINFORCE recommender system. En *Proceedings of WSDM '19* (pp. 456–464). ACM.

Dulac-Arnold, G., Evans, R., van Hasselt, H., Sunehag, P., Lillicrap, T., Hunt, J., Mann, T., Weber, T., Degris, T., & Coppin, B. (2015). Deep reinforcement learning in large discrete action spaces. *arXiv:1512.07679*.

Huang, J., Oosterhuis, H., de Rijke, M., & van Hoof, H. (2020). Keeping dataset biases out of the simulation: A debiased simulator for reinforcement learning based recommender systems. En *Proceedings of the 14th ACM Conference on Recommender Systems* (pp. 190–199). ACM. https://doi.org/10.1145/3383313.3412252

Ie, E., Hsu, C., Mladenov, M., Jain, V., Narvekar, S., Wang, J., Wu, R., & Boutilier, C. (2019). RecSim: A configurable simulation platform for recommender systems. *arXiv:1909.04847*.

Marlin, B. M., Zemel, R. S., Roweis, S., & Slaney, M. (2007). Collaborative filtering and the missing at random assumption. En *Proceedings of UAI 2007* (pp. 267–275).

Marlin, B. M., & Zemel, R. S. (2009). Collaborative prediction and ranking with non-random missing data. En *Proceedings of RecSys '09* (pp. 5–12). ACM.

Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.

Rohde, D., Bonner, S., Dunlop, T., Vasile, F., & Karatzoglou, A. (2018). RecoGym: A reinforcement learning environment for the problem of product recommendation in online advertising. *arXiv:1808.00720*.

Schnabel, T., Swaminathan, A., Singh, A., Chandak, N., & Joachims, T. (2016). Recommendations as treatments: Debiasinglearning and evaluation. En *Proceedings of ICML 2016* (pp. 1670–1679).

Shi, B., Ozsoy, M. G., Hurley, N., Smyth, B., Tragos, E. Z., Geraci, J., & Lawlor, A. (2019). PyRecGym: A reinforcement learninggym forrecommender systems. En *Proceedings ofRecSys '19* (pp. 491–495). ACM.

Shi, J.-C., Yu, Y., Da, Q., Chen, S.-Y., & Zeng, A.-X. (2019). Virtual-Taobao: Virtualizing real-world online retail environmentfor reinforcementlearning. En *Proceedingsof AAAI 2019* (pp. 4902–4909).

Wang, H., Chen, Z., Wang, H., Tan, Y., Pan, L., Liu, T., ... Lin, Z. (2025). Unbiasedrecommender learningfrom implicitfeedback viaweakly supervisedlearning. En *Proceedingsof ICML 2025*.

Wang, X., Zhang, R., Sun, Y., & Qi, J. (2019). Doublyrobust jointlearning forrecommendation on data missing not at random. En *Proceedingsof ICML 2019* (pp. 6638–6647).

Zheng, G., Zhang, F., Zheng, Z., Xiang, Y., Yuan, N. J., Xie, X., & Li, Z. (2018). DRN: A deep reinforcementlearning frameworkfor newsrecommendation. En *Proceedingsof WWW 2018* (pp. 167–176).

Zou, L., Xia, L., Ding, Z., Song, J., Liu, W., & Yin, D. (2019). Reinforcementlearning tooptimize long-termuser engagement in recommender systems. *arXiv:1902.05570*.

KEEPING DATASET BIASES OUT OF THE SIMULATION

A Debiased Simulator for Reinforcement Learning based Recommender Systems

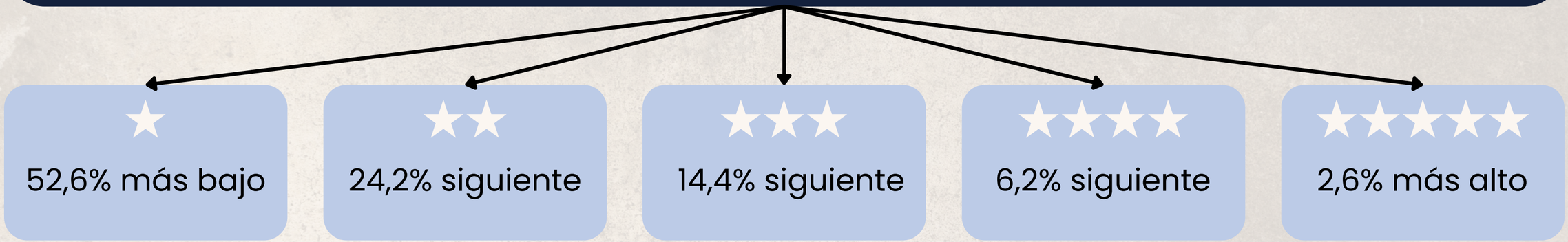
Jin Huang · Harrie Oosterhuis · Maarten de Rijke · Herke van Hoof

University of Amsterdam · RecSys '20, 14th ACM Conference on Recommender Systems

Presentado por: Darío Donoso y Josefina Nazar

Dataset Sintético: Construcción

MCAR Se muestrean 20 números al azar de N(0,1). 10 definen p_u (para cada usuario) y 10 definen q_i (para cada ítem). Producto punto define las preferencias (Zou et al., 2019).



MNAR

- La observación uniforme es del 5%.
- Con sesgo de positividad, P(o = 1 | y) = [2,9%; 2,1%; 3,5%; 16%; 57,7%] para y = 1...5, para obtener una distribución de ratings similar a Yahoo!R3.
- La intensidad del sesgo la controla α. Se prueban los valores α = 0,0; 0,2; 0,4; 0,6; 0,8 y 1,0.

$$P(o_{u,i} \mid y_{u,i}) = \alpha\, P(o_{u,i} \mid y_{u,i}, \text{pos}) + (1-\alpha)\, P(o_{u,i} \mid \text{unif})$$

Deep-Q-Network (DQN)

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Fig. 11. Mnih et al., 2013

Resultados en Métricas

Method	Yahoo!R3				Coat				Synthetic ($\alpha = 1$)			
	MSE↓	MAE↓	ACC↑	Click-ACC↑	MSE↓	MAE↓	ACC↑	Click-ACC↑	MSE↓	MAE↓	ACC↑	Click-ACC↑
MF-IPS	1.518	0.999	0.336	0.889	1.129	0.878	0.311	0.827	1.445	0.997	0.284	0.901
MF-Naive	2.263	1.287	0.222	0.761	1.217	0.914	0.287	0.830	1.780	1.093	0.273	0.856

Fig. 11. Huang et al., RecSys '20, 2020